System Design: Netflix

Video Streaming Platform — Senior Engineer Interview Walkthrough

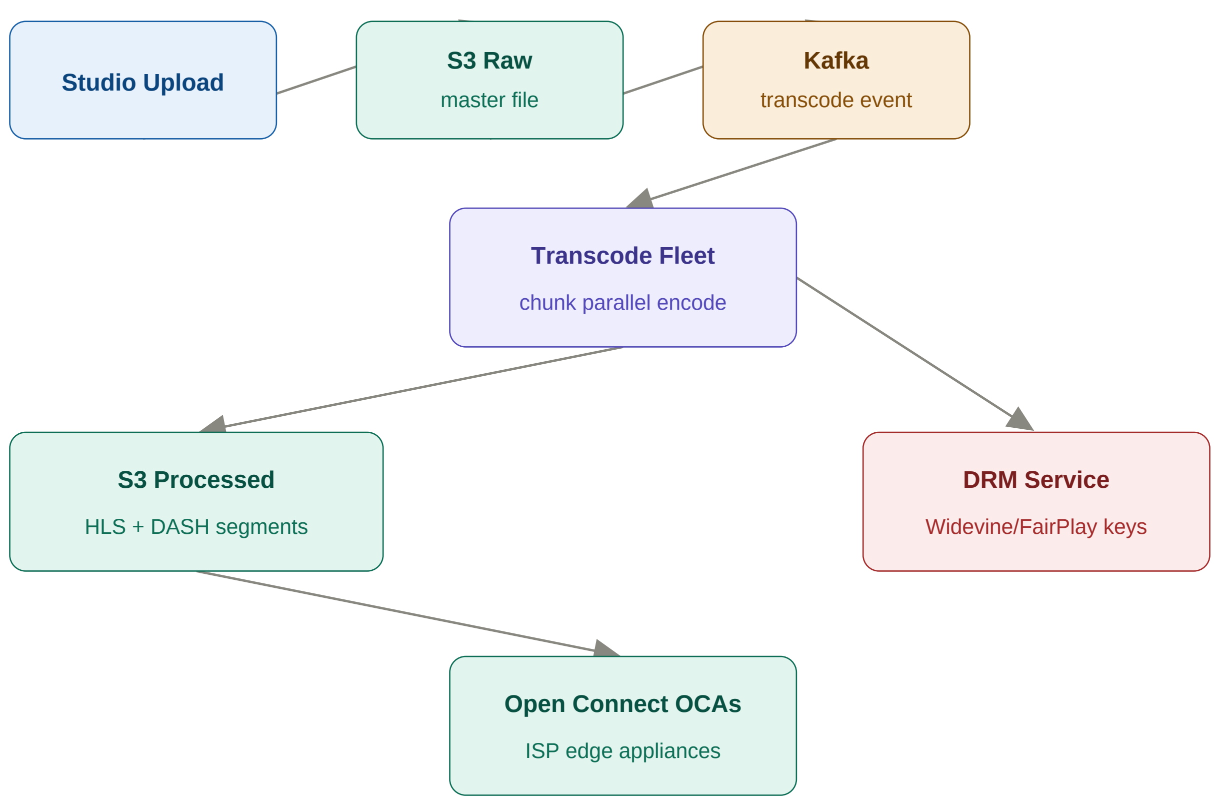
1. Requirements & Capacity

Think out loud: "I'll scope to: playback path, video processing, and recommendations. 220M subscribers, 60M concurrent streams at peak, 1.5 PB content library."

Scale	Users/RPS	Architecture	Storage
Low	10k users 100 streams	Origin server + S3 + basic CDN	~1 TB video
Mid	1M users 50k streams	Multi-region origin + CloudFront + adaptive bitrate	~100 TB
High	220M users 60M concurrent	Open Connect CDN + AWS multi-region + AV1 encoding	~1.5 PB

2. Architecture — Video Upload & Transcode

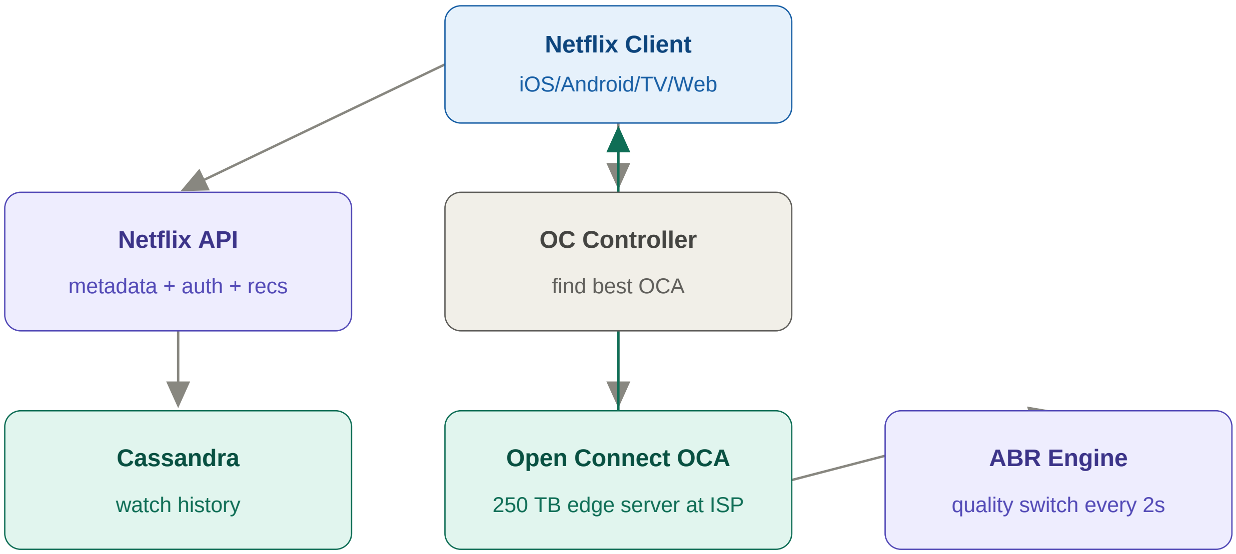
Think out loud: "Before users watch, content must be transcoded into 6+ profiles. The key insight: split the video into chunks and transcode in parallel — 720 chunks for a 2-hour film transcoded simultaneously = 100x faster."



Upload & transcode path: studio → S3 raw → Kafka → parallel transcode fleet → S3 processed segments → replicated to Open Connect edge appliances at ISPs.

3. Architecture — Playback Path

Think out loud: "User presses Play. The client asks Open Connect controller for the nearest OCA with the content. Client fetches HLS manifest directly from OCA. Never touches Netflix origin. Adaptive bitrate switches quality every 2 seconds."



yback path: client asks OC Controller for nearest OCA → streams directly from ISP-local OCA. Netflix origin never handles video bytes. ABR switches quality transparent to u

4. DB Schema

titles		
PK	**title_id**	*BIGINT*
	name	*VARCHAR(256)*
	type	*ENUM*
	genre	*VARCHAR[]*
	rating	*DECIMAL(3,1)*
	release_year	*INT*

video_assets		
PK	**asset_id**	*BIGINT*
FK	title_id	*BIGINT*
	profile	*VARCHAR*
	manifest_url	*TEXT*
	duration_sec	*INT*
	status	*ENUM*

watch_history		
PK	**user_id**	*BIGINT*
PK	**title_id**	*BIGINT*
	watched_at	*TIMESTAMP*
	position_sec	*INT*
	completed	*BOOLEAN*

user_profiles		
PK	**user_id**	*BIGINT*
	taste_vector	*FLOAT[]*
	genre_weights	*JSONB*
	updated_at	*TIMESTAMP*

Schem … *query).*

Why this choice: watch_history in Cassandra. Partition key = user_id, clustering = watched_at DESC. "Continue watching" query = LIMIT 20 from user's partition = single disk read. At 220M users with daily events = billions of rows, Cassandra is the only choice.

Why build own CDN: Open Connect proprietary CDN: saves ~$1B/year vs Akamai. More importantly, they optimise for large sequential video reads (not small web objects), control caching policy per title, and can pre-load popular content proactively during off-peak hours.

5. API & Trade-offs

GET	`/api/v1/titles?query=&genre;=&page;=`	Browse + search catalogue
GET	`/api/v1/play/{title_id}?profile=auto`	Signed manifest URL + DRM licence URL
POST	`/api/v1/watch-events`	{title_id, position_sec, event: PAUSE/RESUME/STOP}
GET	`/api/v1/continue-watching`	Top 20 in-progress titles
GET	`/api/v1/recommendations`	ML-ranked personalised titles

Decision	Choice	Gain	Trade-off
CDN	Open Connect	Cost, video-optimised	Massive infra investment
Streaming	Adaptive bitrate	Smooth on bad networks	Complex manifest mgmt
Watch history	Cassandra	Petabyte scale	Eventual consistency
Transcode	Parallel chunks	100x faster processing	Complex orchestration